

Defense Study Book

Machine Learning & Perception ■ LSTM (Hochreiter) ■ rCFD and CFD

Version 2 ■ October 2026

How this book works

Every topic has four layers. Read them in this order; the first two take about two minutes.

1. **2-minute summary** – the whole topic on half a page: the one idea, the one formula, the words the examiner wants to hear.
2. **Picture it** – an everyday image (an analogy). You remember a picture much longer than a sentence, and it gives you words when you are nervous.
3. **Board drill** – the formula you must be able to write on the whiteboard. Write it by hand three times, without looking. Hands learn faster than eyes (tactile, or kinaesthetic, learning), and on the day you will stand at a board.
4. **Questions** – predicted exam questions. **Say** is what you say out loud, **Write** is what you put on the board, **Follow-up** is the next question that usually comes.

Who asks what (from the list of past students):

- Prof. Klambauer: empirical risk, loss, gradient descent, backprop and the delta error, generalization. “Write the simplest version first.”
- Prof. Hochreiter: delta error, quick questions (regularization, optimizers, activations, the activation from Linz), REINFORCE, and everything LSTM.
- Others (Rekabsaz, Lehner, Schlüter): Transformers and attention, RNNs, SVM, VC dimension, universal approximation.
- Prof. Pirker (elective track): rCFD, your curves, CFD basics. Chapter 7.

Study plan (suggested). Day 1: Chapters 1–2 plus all board drills. Day 2: Chapters 3–4. Day 3: Chapters 5–6. Day 4: Chapters 7–8 and the thesis. Then repeat only the summaries and board drills every morning.

Contents

1 Empirical risk minimization	3
2 Backpropagation and the delta error	4
3 Training: optimizers, activations, regularization	6
4 Recurrent networks and LSTM (Prof. Hochreiter)	7
4.1 Recurrent networks and BPTT	7
4.2 LSTM	9
4.3 Beyond LSTM: xLSTM, Mamba, Hopfield	10
5 Attention and Transformers	11
6 Classical ML asked in the same slot	12
7 rCFD and CFD (Prof. Pirker)	14

8 Predicted questions and rapid-fire drill	15
8.1 Thesis questions (first 5 minutes)	15
8.2 Rapid-fire (say it in one breath)	16

1 Empirical risk minimization

2-minute summary ■ ERM (Klambauer's favourite first question)

One idea. We want a model that is good on *future* data. We cannot measure that, because the true data distribution is unknown. So we measure the average error on the data we *have* and make it small. That is empirical risk minimization.

Four ingredients: data $Z = \{(\mathbf{x}_n, y_n)\}_{n=1}^N$ • model $g(\mathbf{x}; \mathbf{w})$ • loss $L(y, \hat{y})$ • average + minimize.

$$R_{\text{emp}}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N L(y_n, g(\mathbf{x}_n; \mathbf{w})), \quad R_{\text{gen}}(g) = \mathbb{E}_{(\mathbf{x}, y) \sim p} [L(y, g(\mathbf{x}; \mathbf{w}))].$$

Key words: i.i.d. data • unknown $p(\mathbf{x}, y)$ • generalization error • test set • cross-validation • overfitting.

Picture it. A driving school. The *generalization error* is how well you drive on any road in Austria, which nobody can test completely. The *empirical risk* is your score on the practice course. You train on the practice course (minimize R_{emp}), and the driving test on new roads (test set) estimates how you will really drive. Memorizing every bump of the practice course is overfitting.

Board drill (write it by hand 3 times). $Z = \{(\mathbf{x}_n, y_n)\}$ $\hat{y} = g(\mathbf{x}; \mathbf{w})$ $R_{\text{emp}} = \frac{1}{N} \sum_n L(y_n, g(\mathbf{x}_n; \mathbf{w}))$ $\hat{\mathbf{w}} = \arg \min_{\mathbf{w}} R_{\text{emp}}$ $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} R_{\text{emp}}.$

Q1.1 Write down a supervised data set, a model and the empirical risk.

Klambauer

Say: A data set is N pairs of input and label, drawn i.i.d. from an unknown distribution. The model maps an input to a prediction. The loss compares prediction and label. The empirical risk is the mean loss over the training set, and training minimizes it.

Write:

$Z = \{(\mathbf{x}_n, y_n)\}_{n=1}^N$, $\mathbf{x}_n \in \mathbb{R}^D$; $\hat{y}_n = g(\mathbf{x}_n; \mathbf{w})$; $R_{\text{emp}}(\mathbf{w}) = \frac{1}{N} \sum_n L(y_n, \hat{y}_n).$

Follow-up: “Is this only for neural networks?” No: g can be a linear model, a random forest or a kernel machine. Only the inside of g changes.

Source: DLNN script WS24, Ch. 3 and 5.1, PDF p. 23–26, 60.

Q1.2 Difference between empirical risk and generalization error?

Klambauer

Say: The generalization error is the expected loss over the true distribution: the real quality on new data. We cannot compute it because p is unknown. The empirical risk is its estimate on the training data, and it is biased low because we chose $\mathbf{w}$ on those data.

Write:

$R_{\text{gen}} = \int L(y, g(\mathbf{x}; \mathbf{w})) p(\mathbf{x}, y) d\mathbf{x} dy \approx \frac{1}{M} \sum_{m \in \text{test}} L(y_m, g(\mathbf{x}_m; \mathbf{w})).$

Follow-up: “How do you estimate it?” Test set (law of large numbers), or L -fold cross-validation, which is almost unbiased and uses every point once for testing.

Source: DLNN script WS24, Sec. 5.2–5.3, PDF p. 60–64.

Q1.3 Which loss functions do you know? Which properties do they need?

Klambauer

Say: A loss is non-negative and zero for a perfect prediction. For gradient training it must be differentiable. The 0–1 loss is not, so we train on a surrogate such as cross-entropy or the hinge loss.

Write:

Squared $\frac{1}{2}(y - \hat{y})^2$ • binary CE $-y \log \hat{y} - (1 - y) \log(1 - \hat{y})$ • categorical CE $-\sum_k y_k \log \hat{y}_k$ • 0–1 $\mathbb{I}[y \neq \hat{y}]$ • hinge $\max(0, 1 - y g(\mathbf{x}))$.

Follow-up: “Why squared loss for regression?” Under Gaussian noise, minimizing it equals maximizing the likelihood.

Source: DLNN script WS24, Sec. 4.1.2, Table 4.1, Eqs. (4.8)–(4.11).

Q1.4 Show it on linear regression.

Klambauer

Say: Linear model, squared loss. The risk is convex, so there is one minimum: closed form or gradient descent.

Write:

$$R_{\text{emp}} = \frac{1}{2} \|\mathbf{y} - X\mathbf{w}\|^2, \quad \hat{\mathbf{w}} = (X^\top X)^{-1} X^\top \mathbf{y}, \quad \nabla_{\mathbf{w}} R_{\text{emp}} = X^\top (X\mathbf{w} - \mathbf{y}).$$

Follow-up: For one sample the gradient is $(\mathbf{w}^\top \mathbf{x} - y) \mathbf{x}$: *error times input*. Remember this shape; backprop has exactly the same shape, $\delta_i a_j$.

Source: DLNN script WS24, Eqs. (4.12)–(4.15), PDF p. 31.

Q1.5 Overfitting, underfitting, bias–variance, double descent.

Klambauer

Say: Too simple: underfitting, high bias. Too complex: overfitting, high variance, low training error but high test error. Classical picture: U-shaped test error. Large networks: double descent, the test error falls again after the interpolation point.

Write:

$$\mathbb{E}[(y - g)^2] = \sigma_{\text{noise}}^2 + \text{bias}^2 + \text{variance}.$$

Picture it. Darts. Bias: all darts land together but off the centre. Variance: the darts scatter around the centre.

Source: DLNN script WS24, Sec. 5.4, PDF p. 65.

2 Backpropagation and the delta error

2-minute summary ■ Backprop (asked by Klambauer AND Hochreiter)

One idea. To learn, every weight needs its gradient. Backprop gets all of them with one forward pass and one backward pass. The trick is the *delta error*: how much the loss changes when the pre-activation of one neuron changes. Deltas are computed at the output and passed *down* layer by layer.

Three lines to know by heart:

$$\underbrace{\delta_i = \frac{\partial L}{\partial s_i}}_{\text{definition}} \quad \underbrace{\frac{\partial L}{\partial w_{ij}} = \delta_i a_j}_{\text{gradient}} \quad \underbrace{\delta_j = f'(s_j) \sum_i \delta_i w_{ij}}_{\text{recursion}}$$

Output with a matching pair (softmax + cross-entropy, sigmoid + BCE, linear + squared): $\delta^{[L]} = \hat{\mathbf{y}} - \mathbf{y}$.

Key words: forward pass stores s and a • chain rule • delta • outer product • $O(W)$ • vanishing / exploding gradients (Hochreiter 1991).

Picture it. A football team that lost 3–0. The coach (loss) starts at the goal that was conceded and walks *backwards* through the team. Each player’s share of the blame (delta) is: the blame of the players he passed to, weighted by how strongly he passed to them (w_{ij}), times how awake he was himself ($f'(s_j)$). A tired player ($f' \approx 0$, saturated sigmoid) passes no blame on: that is the vanishing gradient.

Board drill (write it by hand 3 times). Forward: $s_i = \sum_j w_{ij} a_j$, $a_i = f(s_i)$. Backward: $\delta_k = \hat{y}_k - y_k$ (output), $\delta_j = f'(s_j) \sum_i \delta_i w_{ij}$ (hidden). Update: $\Delta w_{ij} = -\eta \delta_i a_j$. Matrix form: $\delta^{[l-1]\top} = \delta^{[l]\top} W^{[l]} \text{diag}(f'(s^{[l-1]}))$, $\partial L / \partial W^{[l]} = \mathbf{a}^{[l-1]} \delta^{[l]\top}$.

Q2.1 Derive the gradient for one weight. Define the delta.

Klambauer, Hochreiter

Say: By the chain rule the loss depends on w_{ij} only through s_i . The derivative of s_i with respect to w_{ij} is just a_j . What remains, $\partial L / \partial s_i$, we call the delta.

Write:

$$\frac{\partial L}{\partial w_{ij}} = \frac{\partial L}{\partial s_i} \frac{\partial s_i}{\partial w_{ij}} = \delta_i a_j, \quad \text{since } s_i = \sum_j w_{ij} a_j \Rightarrow \partial s_i / \partial w_{ij} = a_j.$$

Follow-up: “Interpret it.” Gradient = error signal above $\times$ activity below. The same shape as linear regression: error $\times$ input.

Source: DLNN script WS24, Eqs. (4.63)–(4.68), PDF p. 45.

Q2.2 Derive the recursive formula for the delta of a hidden unit.

Klambauer, Hochreiter

Say: A hidden unit j influences the loss through all units i it feeds. So I sum over them with the chain rule.

Write:

$$\delta_j = \frac{\partial L}{\partial s_j} = \sum_i \frac{\partial L}{\partial s_i} \frac{\partial s_i}{\partial s_j} = \sum_i \delta_i w_{ij} f'(s_j), \quad \text{because } s_i = \sum_j w_{ij} f(s_j).$$

Follow-up: Past students got stuck on indices; Prof. Hochreiter helped. Keep i = layer above, j = layer below, w_{ij} = from j to i .

Source: DLNN script WS24, Eq. (4.71), PDF p. 46.

Q2.3 Delta at the output for softmax + cross-entropy?

Hochreiter

Say: Simply prediction minus label, $\delta = \hat{\mathbf{y}} - \mathbf{y}$. The derivative of the log inside the cross-entropy cancels the derivative of the softmax.

Write:

$$L = -\sum_k y_k \log \hat{y}_k, \quad \hat{y}_k = \frac{e^{s_k}}{\sum_m e^{s_m}}, \quad \frac{\partial \hat{y}_k}{\partial s_i} = \hat{y}_k (\delta_{ki} - \hat{y}_i) \Rightarrow \frac{\partial L}{\partial s_i} = -\sum_k y_k (\delta_{ki} - \hat{y}_i) = \hat{y}_i - y_i \quad (\text{using } \sum_k y_k = 1).$$

Follow-up: Same result for sigmoid + BCE and linear + squared loss: that is why these pairs belong together.

Source: DLNN script WS24, Table 4.1, Eq. (4.78).

Q2.4 Vanishing and exploding gradients: where do they come from?

Hochreiter

Say: Each layer multiplies the delta by $W^{[l]} \text{diag}(f'(s))$. After many layers this is a product of many matrices, so the norm shrinks or grows exponentially. For the sigmoid $|f'| \leq 0.25$, so it shrinks. Sepp Hochreiter showed this in 1991 in Linz.

Write:

$$\|\delta^{[l-k]}\| \lesssim (\max |f'| \cdot \|W\|)^k \|\delta^{[l]}\|.$$

Follow-up: “Remedies?” ReLU / SELU, careful initialization (keep the Jacobian norm near 1), normalization, residual connections, LSTM for time.

Source: DLNN script WS24, box after Eq. (4.84), PDF p. 52–53.

Q2.5 How is a neural network built? What is one neuron?

Klambauer

Say: A neuron takes a weighted sum of its inputs plus a bias, the pre-activation, and applies a non-linear activation. Neurons form layers; layers are stacked; the last layer gives the output.

Write:

$$s = \mathbf{w}^\top \mathbf{a} + b, \quad a = f(s); \quad \text{layer: } \mathbf{a}^{[l]} = f(W^{[l]} \mathbf{a}^{[l-1]} + \mathbf{b}^{[l]}).$$

Follow-up: “And without activation functions?” Then all layers collapse into one linear map $W^{[L]} \dots W^{[1]} = W$. It cannot even solve XOR.

Source: DLNN script WS24, Eqs. (3.2)–(3.3), Sec. 4.5.

3 Training: optimizers, activations, regularization

2-minute summary ■ The rapid-fire block (Hochreiter asks these fast)

Optimizers: GD, SGD, momentum, AdaGrad, RMSprop, Adam. **Activations:** sigmoid, tanh, ReLU, Leaky ReLU, ELU, SELU. **From Linz:** ELU (Clevert, Unterthiner, Hochreiter) and SELU (Klambauer et al. 2017). **Regularization:** weight decay, dropout, early stopping, data augmentation, noise, Flat Minimum Search (Hochreiter). **Output layer:** linear (regression), sigmoid (binary), softmax (multi-class). **Why mini-batches:** good gradient estimate, fast on GPU, useful noise.

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla R_{\text{emp}}, \quad \text{Adam: } \mathbf{w} \leftarrow \mathbf{w} - \eta \frac{\hat{\mathbf{m}}}{\sqrt{\hat{\mathbf{v}} + \epsilon}}, \quad \text{SELU: } \lambda \begin{cases} x & x > 0 \\ \alpha(e^x - 1) & x \leq 0 \end{cases}$$

with $\lambda \approx 1.0507$, $\alpha \approx 1.6733$.

Picture it. Walking down a foggy mountain. Gradient descent: look at the slope under your feet and step downhill. Momentum: a heavy ball that keeps rolling through small bumps. Adam: shoes that adjust their step to how steep and how noisy each direction is. Mini-batch: asking a few people for the way instead of everybody (too slow) or one person (too noisy).

Board drill (write it by hand 3 times). GD $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla R_{\text{emp}}$ • momentum $\mathbf{v} \leftarrow \mu \mathbf{v} - \eta \nabla R_{\text{emp}}$, $\mathbf{w} \leftarrow \mathbf{w} + \mathbf{v}$ • weight decay $R_{\text{emp}} + \frac{\lambda}{2} \|\mathbf{w}\|^2$ • dropout: keep unit with prob. $1 - p$ • batch norm $\hat{s} = (s - \mu_B) / \sqrt{\sigma_B^2 + \epsilon}$, then $\gamma \hat{s} + \beta$.

Q3.1 Which activation function comes from Linz? What does it do?

Hochreiter

Say: SELU, the scaled exponential linear unit, from Klambauer, Unterthiner, Mayr and Hochreiter (2017); and before it ELU. SELU makes networks *self-normalizing*: mean and variance of the activations are pulled to (0, 1) layer by layer, without batch normalization.

Write:

The map of (mean, variance) from one layer to the next has the fixed point (0, 1); its Jacobian there has spectral norm $0.7877 < 1$, so the fixed point is stable (a contraction).

Picture it. A thermostat: whatever the temperature, the layer pushes it back to the set point.

Source: DLNN script WS24, Sec. 10.4, PDF p. 137–141.

Q3.2 Which optimizers do you know? Are there methods other than gradient descent?

Hochreiter

Say: GD, SGD, momentum, AdaGrad, RMSprop, Adam. Other methods: closed form (linear models), IRLS, RProp and Quickprop, second-order methods (Newton, Gauss–Newton, Levenberg–Marquardt, conjugate gradients, natural gradient), and gradient-free search (evolutionary algorithms).

Source: DLNN script WS24, Ch. 7, PDF p. 85–101; SS25 script “Higher Order Optimization”.

Q3.3 Which regularization techniques do you know?

Hochreiter

Say: Weight decay, dropout, early stopping, data augmentation, input and weight noise, weight sharing, multi-task learning, pruning (Optimal Brain Damage and Surgeon), and Flat Minimum Search, which looks for wide minima that generalize better.

Write:

Weight decay gradient: $\nabla R_{\text{emp}} + \lambda \mathbf{w}$ (the weights shrink each step).

Follow-up: “Why does dropout work?” It trains an ensemble of thinned networks and stops units from co-adapting.

Source: DLNN script WS24, Ch. 8, PDF p. 103–119.

Q3.4 How can you do feature selection?

Hochreiter

Say: Three families. Filter: rank features by a score like correlation or mutual information. Wrapper: search subsets with the model, e.g. forward or backward selection. Embedded: the model selects while it trains, e.g. L1 regularization (Lasso) pushes weights to exactly zero.

Q3.5 Why initialize randomly? What does good initialization do?

Klambauer

Say: Equal weights make all units in a layer learn the same thing, so we break the symmetry with random weights. The scale should keep the variance of activations forward and of deltas backward constant, so nothing vanishes or explodes.

Write:

$\text{Var}(w) = 1/n_{\text{in}}$ (LeCun), $2/n_{\text{in}}$ (He, ReLU), $2/(n_{\text{in}} + n_{\text{out}})$ (Glorot).

Source: DLNN script WS24, Ch. 9, PDF p. 123–129.

Q3.6 Universal approximation theorem.

Lehner

Say: One hidden layer with finitely many units and a non-polynomial activation can approximate any continuous function on a compact set arbitrarily well. It says such a network exists; it does not say how many units, nor that gradient descent finds it.

Source: DLNN I+II script SS25, “Deep Learning theory”, PDF p. 321.

Q3.7 CNNs: why do they work for images? What is ResNet?

Klambauer

Say: Local filters with shared weights slide over the image: few parameters, translation equivariance. Pooling reduces size and adds invariance. Depth grows the receptive field. ResNet adds identity skips, $\mathbf{a}^{[l+1]} = \mathbf{a}^{[l]} + F(\mathbf{a}^{[l]})$, so the Jacobian is $I + \partial F$ and gradients pass through hundreds of layers.

Write:

Output size $\lfloor (n + 2p - k)/s \rfloor + 1$.

Source: DLNN script WS24, Ch. 6; SS25 script PDF p. 176, 323.

4 Recurrent networks and LSTM (Prof. Hochreiter)

This is Prof. Hochreiter’s own field: he found the vanishing gradient (1991) and invented LSTM with Schmidhuber (1997). Expect him to go deep and to enjoy it. Notation of his lecture: input $\mathbf{x}(t)$, pre-activation $\mathbf{s}(t)$, activation $\mathbf{a}(t)$ or hidden state $\mathbf{h}(t)$, memory cell $\mathbf{c}(t)$, input weights W , recurrent weights R , output weights V .

4.1 Recurrent networks and BPTT

2-minute summary ■ RNN in two minutes

One idea. An RNN reads a sequence one step at a time and keeps a hidden state: a summary of everything seen so far. The *same* weights are used at every step, so it works for any length. Training = backpropagation through time (BPTT): unroll the net over time and backprop through

the unrolled graph.

$$\mathbf{s}(t) = W\mathbf{x}(t) + R\mathbf{a}(t-1), \quad \mathbf{a}(t) = f(\mathbf{s}(t)), \quad \hat{\mathbf{y}}(t) = V\mathbf{a}(t), \quad L = \sum_{t=1}^T L(t).$$

$$\boldsymbol{\delta}(t) = \text{diag}(f'(\mathbf{s}(t))) \left(V^\top \frac{\partial L(t)}{\partial \hat{\mathbf{y}}(t)} + R^\top \boldsymbol{\delta}(t+1) \right),$$

$$\frac{\partial L}{\partial R} = \sum_t \boldsymbol{\delta}(t) \mathbf{a}(t-1)^\top, \quad \frac{\partial L}{\partial W} = \sum_t \boldsymbol{\delta}(t) \mathbf{x}(t)^\top.$$

Key words: weight sharing in time • Turing complete • BPTT • truncated BPTT • RTRL • vanishing gradient in time.

Picture it. Reading a novel. You do not remember every word, you keep a running summary in your head (the hidden state) and update it with each new sentence, using the same reading skill (the same weights) on every page. BPTT is re-reading the book backwards to find which pages caused your wrong guess about the murderer.

Board drill (write it by hand 3 times). $\mathbf{s}(t) = W\mathbf{x}(t) + R\mathbf{a}(t-1)$, $\mathbf{a}(t) = f(\mathbf{s}(t))$, $\hat{\mathbf{y}}(t) = V\mathbf{a}(t)$; $\boldsymbol{\delta}(t) = \text{diag}(f'(\mathbf{s}(t))) (V^\top \partial L(t)/\partial \hat{\mathbf{y}}(t) + R^\top \boldsymbol{\delta}(t+1))$; $\partial L/\partial R = \sum_t \boldsymbol{\delta}(t) \mathbf{a}(t-1)^\top$. Note: $\boldsymbol{\delta}(t+1)$, the future, not $t-1$. Gradient uses $\mathbf{a}(t-1)$, not $\mathbf{s}(t-1)$.

Q4.1 Which recurrent architectures do you know?

Hochreiter

Say: Jordan network: feeds the *output* of $t-1$ back as input. Elman network: feeds the *hidden* state back, originally with fixed recurrent weights and trained locally in time. Fully recurrent network: trainable recurrent matrix R , every hidden unit sees all others. NARX: shortcuts to outputs further in the past (delays), which fights vanishing gradients only for a finite number of steps. Time-delay networks (TDNN): a fixed window, basically a 1D convolution. Then LSTM and GRU with gates.

Follow-up: “Jordan training trick?” Teacher forcing: during training feed the true label of $t-1$ instead of the prediction.

Source: LSTM lecture WS24, Exam_LSTM summary.pdf.

Q4.2 BPTT, truncated BPTT and RTRL: compare them.

Hochreiter

Say: BPTT: forward through the whole sequence, then backward; exact gradient; memory grows linearly with the length T ; cost $O(T \cdot W)$. Truncated BPTT: backprop only k steps into the past; cheaper, but the gradient is an approximation and long dependencies are cut. RTRL: carries the derivative of the state with respect to all weights forward in time, so it is online and memory does not grow with T , but it is very expensive: $O(N^3)$ numbers stored and $O(N^4)$ operations per step for N hidden units.

Write:

RTRL recursion: $\frac{\partial \mathbf{s}(t)}{\partial w} = \frac{\partial \mathbf{s}(t)}{\partial w} \Big|_{\text{direct}} + \frac{\partial \mathbf{s}(t)}{\partial \mathbf{s}(t-1)} \frac{\partial \mathbf{s}(t-1)}{\partial w}.$

Picture it. BPTT is watching the whole match recording afterwards. RTRL is a coach who updates his notes live during the match: no recording needed, but he must write very fast.

Source: LSTM lecture WS24, BPTT/RTRL exam questions.

Q4.3 Why can a plain RNN not learn long-term dependencies?

Hochreiter

Say: The delta is multiplied at every time step by the recurrent Jacobian $R^\top \text{diag}(f'(\mathbf{s}))$. Over k steps this is a product of k factors, so it decays (or explodes) exponentially in k . After 10–20 steps the error from the past is practically zero.

Write:

$$\left\| \frac{\partial \mathbf{a}(t)}{\partial \mathbf{a}(t-k)} \right\| = \left\| \prod_{\tau=t-k+1}^t \text{diag}(f'(\mathbf{s}(\tau))) R \right\| \leq (\max |f'| \|R\|)^k.$$

Follow-up: “Gradient clipping?” Only fixes *exploding*: it rescales the gradient norm to a threshold, keeps the direction. It does nothing for vanishing.

4.2 LSTM

2-minute summary ■ LSTM in two minutes

One idea. Give the network a *memory cell* that is updated by *adding*, not by multiplying through a squashing function. Then the error can flow back through time unchanged: the *constant error carousel* (CEC). Gates, small sigmoid switches between 0 and 1, decide what to write (input gate), what to keep (forget gate) and what to show (output gate).

$$\begin{aligned} \mathbf{z}(t) &= \tanh(W_z \mathbf{x}(t) + R_z \mathbf{h}(t-1) + \mathbf{b}_z) && \text{cell input} \\ \mathbf{i}(t) &= \sigma(W_i \mathbf{x}(t) + R_i \mathbf{h}(t-1) + \mathbf{b}_i) && \text{input gate} \\ \mathbf{f}(t) &= \sigma(W_f \mathbf{x}(t) + R_f \mathbf{h}(t-1) + \mathbf{b}_f) && \text{forget gate} \\ \mathbf{o}(t) &= \sigma(W_o \mathbf{x}(t) + R_o \mathbf{h}(t-1) + \mathbf{b}_o) && \text{output gate} \\ \mathbf{c}(t) &= \mathbf{f}(t) \odot \mathbf{c}(t-1) + \mathbf{i}(t) \odot \mathbf{z}(t) && \text{memory update} \\ \mathbf{h}(t) &= \mathbf{o}(t) \odot \tanh(\mathbf{c}(t)) && \text{output} \end{aligned}$$

Why it works: $\frac{\partial \mathbf{c}(t)}{\partial \mathbf{c}(t-1)} = \text{diag}(\mathbf{f}(t))$, which is $\approx I$ when the forget gate is open. The original 1997 LSTM had no forget gate: $\mathbf{c}(t) = \mathbf{c}(t-1) + \mathbf{i} \odot \mathbf{z}$, derivative exactly I . The forget gate was added by Gers, Schmidhuber and Cummins (2000).

Picture it. A conveyor belt (the cell state) running through a factory. Things ride on it without being changed. Three workers stand next to it: one decides what to put *on* the belt (input gate), one decides what to throw *off* (forget gate), one decides what to *show* to the next department (output gate). Because the belt does not squash anything, an item put on at the beginning is still there at the end: long-term memory.

Board drill (write it by hand 3 times). $\mathbf{c}(t) = \mathbf{f} \odot \mathbf{c}(t-1) + \mathbf{i} \odot \mathbf{z}$ $\mathbf{h}(t) = \mathbf{o} \odot \tanh(\mathbf{c}(t))$ gates $= \sigma(W\mathbf{x}(t) + R\mathbf{h}(t-1) + \mathbf{b})$, cell input $= \tanh(\cdot)$ $\partial \mathbf{c}(t)/\partial \mathbf{c}(t-1) = \text{diag}(\mathbf{f}(t)) \approx I$. Draw the cell: belt on top, three σ gates, one $\tanh$ in, one $\tanh$ out.

Q4.4 Explain the constant error carousel.

Hochreiter

Say: In the original LSTM the cell connects to itself with weight 1 and a linear activation. So the local error in the cell is multiplied by exactly 1 at every step: it neither vanishes nor explodes. The gates protect this carousel: the input gate protects the cell from irrelevant inputs, the output gate protects other units from irrelevant cell content.

Write:

$$\mathbf{c}(t) = \mathbf{c}(t-1) + \mathbf{i}(t) \odot \mathbf{z}(t) \Rightarrow \partial \mathbf{c}(t)/\partial \mathbf{c}(t-1) = I.$$

Source: LSTM lecture WS24, CEC slides; Hochreiter & Schmidhuber (1997).

Q4.5 Why was the forget gate added? What about drift?

Hochreiter

Say: Without a forget gate the cell can only accumulate; on long, continuous input streams the state grows without limit and the cell saturates. The forget gate lets the network reset its memory. Related: *internal state drift*, when inputs are mostly of one sign the state drifts away; the remedy in the original paper is to bias the input gates towards zero at the start (negative bias), so the cell writes only when needed.

Follow-up: “Ticker steps?” Letting the LSTM run extra steps after the input has ended, so it can process what it has stored before it must answer.

Q4.6 LSTM variants: peepholes, CIFG, GRU.

Hochreiter

Say: Peephole connections let the gates see the cell state $\mathbf{c}(t-1)$ directly. CIFG (coupled input and forget gate) sets $\mathbf{f} = 1 - \mathbf{i}$: write only as much as you forget. GRU (Cho et al. 2014) goes further: no separate cell, an update gate that acts as coupled input and forget gate, and a reset gate that decides how much the old state enters the candidate.

Write:

GRU: $\mathbf{u} = \sigma(\cdot)$, $\mathbf{r} = \sigma(\cdot)$, $\tilde{\mathbf{h}} = \tanh(W\mathbf{x}(t) + R(\mathbf{r} \odot \mathbf{h}(t-1)))$, $\mathbf{h}(t) = (1 - \mathbf{u}) \odot \mathbf{h}(t-1) + \mathbf{u} \odot \tilde{\mathbf{h}}$.

Follow-up: Other variants from the lecture: multidimensional LSTM (e.g. 2D images, several time axes), ConvLSTM (convolutions inside the gates, for video).

Q4.7 Dropout in LSTMs: why is it special?

Hochreiter

Say: Naive dropout on the recurrent state at every step multiplies survival probabilities over time, so a memory would survive with probability $(1-p)^T$: long-term memory is destroyed. So we use special forms: dropout only on the non-recurrent connections, variational dropout (the same mask at every time step), or zoneout (randomly *keep* the previous state instead of zeroing units). Dropout is switched off at evaluation.

Q4.8 Bidirectional LSTM, and one label for a whole sequence.

Hochreiter, Schlüter

Say: A bidirectional LSTM runs one LSTM forward and one backward and concatenates the two hidden states, so each step sees past and future; it needs the full sequence, so not for real-time use. For one label per sequence use the last hidden state or global average pooling over all states, then a classifier. Seq2seq: an encoder LSTM reads the input, a decoder LSTM writes the output.

4.3 Beyond LSTM: xLSTM, Mamba, Hopfield

2-minute summary ■ Modern sequence models from Linz and elsewhere

xLSTM (Beck, Hochreiter et al. 2024): (1) *exponential gating* with a normalizer state, so the model can revise a storage decision (a new, more important input can overwrite the old one); (2) sLSTM: scalar memory with memory mixing and a stabilizer against overflow; (3) mLSTM: a *matrix* memory $C \in \mathbb{R}^{d \times d}$ updated by a covariance rule, fully parallelizable; its size does not grow with the sequence length.

$$\text{mLSTM: } C_t = f_t C_{t-1} + i_t \mathbf{v}_t \mathbf{k}_t^\top, \quad \mathbf{n}_t = f_t \mathbf{n}_{t-1} + i_t \mathbf{k}_t, \quad \mathbf{h}_t = \mathbf{o}_t \odot \frac{C_t \mathbf{q}_t}{\max(|\mathbf{n}_t^\top \mathbf{q}_t|, 1)}, \quad i_t = \exp(\tilde{i}_t).$$

Mamba: a *selective* state space model, $\mathbf{h}_t = \bar{A} \mathbf{h}_{t-1} + \bar{B} \mathbf{x}_t$, $\mathbf{y}_t = C \mathbf{h}_t$, where B , C and the step Δ depend on the input; linear in sequence length. Modern Hopfield networks (Ramsauer, Hochreiter et al. 2020): the update $\boldsymbol{\xi}^{\text{new}} = X \text{softmax}(\beta X^\top \boldsymbol{\xi})$ is exactly Transformer attention.

Picture it. Old LSTM memory is a notebook where you write in pencil and can only fade old notes. xLSTM's exponential gate is a highlighter: a new, important note can outshine everything before it. The mLSTM matrix memory is a filing cabinet: store a value under a key ($\mathbf{v}\mathbf{k}^\top$), retrieve it later with a query ($C\mathbf{q}$).

Q4.9 What is new in xLSTM, and why?

Hochreiter

Say: LSTM has three limits: it cannot revise a storage decision, its memory is a small scalar cell, and it cannot be parallelized over time because of memory mixing. xLSTM answers each: exponential gating with normalization revises storage decisions; mLSTM gives a matrix memory with key-value storage; and mLSTM drops memory mixing, so it trains in parallel like a Transformer while inference stays linear in the sequence length.

Write:

$$C_t = f_t C_{t-1} + i_t \mathbf{v}_t \mathbf{k}_t^\top, \text{ retrieval } C_t \mathbf{q}_t; \text{ normalizer } \mathbf{n}_t = f_t \mathbf{n}_{t-1} + i_t \mathbf{k}_t.$$

Source: LSTM lecture WS24, xLSTM exam questions.

Q4.10 What connects Hopfield networks and attention?

Hochreiter

Say: A modern Hopfield network stores patterns X and retrieves the one closest to a query ξ with $\xi^{\text{new}} = X \text{softmax}(\beta X^\top \xi)$. With $\beta = 1/\sqrt{d_k}$ and learned projections this is exactly the attention formula. So attention is associative memory retrieval, and the network can store exponentially many patterns.

5 Attention and Transformers

2-minute summary ■ Attention in two minutes

One idea. Instead of squeezing the past into one hidden state, let every token *look at every other token* and take a weighted mean. Each token makes three vectors by linear maps: a query (what I look for), a key (what I offer), a value (what I give). Weight = softmax of query · key.

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \quad \text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V.$$

Shapes with N tokens: $X \in \mathbb{R}^{N \times d}$, $QK^\top \in \mathbb{R}^{N \times N}$ (rows sum to 1 after softmax), output $\in \mathbb{R}^{N \times d_v}$. **Transformer block:** multi-head self-attention → add & layer norm → position-wise MLP → add & layer norm. **Key words:** scaled dot product • multi-head • positional encoding • permutation equivariance • $O(N^2)$ • encoder/decoder, causal mask.

Picture it. A library. You come with a question (query). Every book has a title on its spine (key) and content inside (value). You compare your question with every title, trust the best matches most (softmax), and read a blend of their contents (weighted sum of values). Multi-head: several librarians, each searching by a different criterion.

Board drill (write it by hand 3 times). $\text{softmax}(QK^\top/\sqrt{d_k})V$; $A_{ij} = \frac{\exp(\mathbf{q}_i^\top \mathbf{k}_j/\sqrt{d_k})}{\sum_m \exp(\mathbf{q}_i^\top \mathbf{k}_m/\sqrt{d_k})}$; $\mathbf{o}_i = \sum_j A_{ij} \mathbf{v}_j$; $\text{MultiHead} = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O$.

Q5.1 Explain the Transformer architecture.

Rekabsaz, Lehner

Say: The input tokens are embedded and get a positional encoding. The encoder is a stack of blocks, each with multi-head self-attention and a feed-forward network, both with residual connections and layer normalization. The decoder has the same blocks, but its self-attention is

masked (a token cannot see the future), and it has a cross-attention layer whose keys and values come from the encoder. BERT uses only the encoder (masked language modelling); GPT only the decoder.

Follow-up: “Why residual + layer norm?” Stable gradients through deep stacks, like ResNet.

Q5.2 Write the matrix multiplications of self-attention.

Klambauer

Say: Three projections of the same input, one $N \times N$ score matrix, a row-wise softmax, and a product with the values.

Write:

$X \in \mathbb{R}^{N \times d}$; $Q, K \in \mathbb{R}^{N \times d_k}$, $V \in \mathbb{R}^{N \times d_v}$; $S = QK^\top / \sqrt{d_k} \in \mathbb{R}^{N \times N}$; $A = \text{softmax}_{\text{row}}(S)$; $O = AV \in \mathbb{R}^{N \times d_v}$.

Follow-up: “Why divide by $\sqrt{d_k}$?” With random unit-variance entries the dot product has variance d_k ; dividing keeps it at 1, so the softmax does not saturate and gradients stay alive.

Q5.3 Permutation invariance: why a problem, how to fix?

Lehner

Say: Self-attention treats the tokens as a set: shuffle the input and the output is shuffled the same way (permutation equivariant). Word order is lost. Fix: add a positional encoding to each token, fixed sinusoids $\sin(p/10000^{2i/d})$, $\cos(\cdot)$, learned position vectors, or relative encodings such as RoPE.

Q5.4 RNN versus Transformer: complexity and trade-offs.

Lehner

Say: RNN: $O(N)$ in sequence length, constant memory for inference, but sequential, so slow to train and long paths between distant tokens. Transformer: $O(N^2)$ time and memory, but fully parallel in training and every pair of tokens is one step apart. xLSTM and Mamba try to get both: parallel training and linear inference.

Write:

RNN $O(N d^2)$ Transformer $O(N^2 d)$.

6 Classical ML asked in the same slot

2-minute summary ■ SVM, kernels, VC, likelihood, REINFORCE

SVM: largest-margin separating hyperplane; soft margin with slack ξ_n and penalty C . **Kernel trick:** the dual needs only dot products, so replace them by $k(\mathbf{x}, \mathbf{x}')$. **VC dimension:** the largest point set a model class can shatter; linear in $\mathbb{R}^d$: $d + 1$. **MLE:** choose the parameters that make the data most likely; overfits with little data. **REINFORCE:** policy gradient by the log-derivative trick.

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_n \xi_n \quad \text{s.t.} \quad y_n(\mathbf{w}^\top \mathbf{x}_n + b) \geq 1 - \xi_n, \quad \xi_n \geq 0; \quad \nabla_\theta J = \mathbb{E} \left[\sum_t \nabla_\theta \log \pi_\theta(a_t | s_t) G_t \right].$$

Picture it. SVM: build the widest possible road between two villages; the houses standing right at the road’s edge are the support vectors, they alone decide where the road goes. Kernel: lift the map into 3D, where a curved border becomes a flat wall, without ever drawing the 3D map.

Board drill (write it by hand 3 times). Primal SVM (above) • dual $\max_\alpha \sum \alpha_n - \frac{1}{2} \sum_{n,m} \alpha_n \alpha_m y_n y_m k(\mathbf{x}_n, \mathbf{x}_m)$, $0 \leq \alpha_n \leq C$, $\sum \alpha_n y_n = 0$ • RBF $k = \exp(-\|\mathbf{x} - \mathbf{x}'\|^2 / 2\sigma^2)$ • $\mathcal{L}(\theta) = \prod_n p(\mathbf{x}_n; \theta)$, $\hat{\theta} = \arg \max \sum_n \log p(\mathbf{x}_n; \theta)$ • REINFORCE loss $-\sum_t \log \pi_\theta(a_t | s_t) (G_t - b)$.

Q6.1 Explain the SVM and write the optimization problem with its constraints.

Kofler, Hochreiter

Say: The SVM separates two classes with the hyperplane of maximal margin $2/\|\mathbf{w}\|$. Maximizing the margin is minimizing $\frac{1}{2}\|\mathbf{w}\|^2$. The soft margin allows violations with slack variables, paid for with C . Only points on or inside the margin get $\alpha_n > 0$: the support vectors.

Write:

(primal and dual as in the board drill); decision $f(\mathbf{x}) = \text{sign}(\sum_n \alpha_n y_n k(\mathbf{x}_n, \mathbf{x}) + b)$.

Follow-up: “Large C ?” Few violations, narrow margin, risk of overfitting. “Small C ?” Wide margin, more violations.

Q6.2 What are kernels, why important, what is the kernel trick?

Kofler

Say: A kernel is a dot product in a feature space, $k(\mathbf{x}, \mathbf{x}') = \langle \phi(\mathbf{x}), \phi(\mathbf{x}') \rangle$. Many algorithms (SVM dual, kernel ridge regression, kernel PCA) use data only through dot products, so we plug in the kernel and get a non-linear method at linear cost, without computing ϕ , which may even be infinite-dimensional (RBF). A valid kernel gives a positive semi-definite Gram matrix (Mercer).

Q6.3 VC dimension: what is it, what is it used for?

Kofler

Say: The largest number of points that the model class can shatter, i.e. classify correctly under every possible labelling. Lines in 2D: 3. It measures capacity and appears in generalization bounds: with probability $1 - \eta$, $R_{\text{gen}} \leq R_{\text{emp}} + \sqrt{\frac{h(\log(2N/h) + 1) - \log(\eta/4)}{N}}$, with h the VC dimension. Small h or large $N \rightarrow$ small gap.

Q6.4 Likelihood, maximum likelihood, expectation maximization.

Widmer, Kofler

Say: The likelihood is the probability of the observed data as a function of the parameters. MLE maximizes it (in practice the log). Problem: with little data it overfits, e.g. the sunrise problem, where an unseen event gets probability zero; a prior (MAP, Laplace smoothing) fixes it. EM is used when there are hidden variables (mixture models, HMMs): the E-step computes the posterior of the hidden variables with the current parameters, the M-step maximizes the expected complete-data log-likelihood; the likelihood never decreases.

Write:

$\hat{\theta}_{\text{MLE}} = \arg \max_{\theta} \sum_n \log p(\mathbf{x}_n; \theta)$; link to ERM: squared loss $\Leftrightarrow$ Gaussian MLE, cross-entropy $\Leftrightarrow$ categorical MLE.

Q6.5 Derive the REINFORCE loss.

Hochreiter

Say: We maximize the expected return over trajectories. The gradient of an expectation is turned into an expectation of a gradient with the log-derivative trick, $\nabla p = p \nabla \log p$. The environment dynamics do not depend on θ , so only the policy terms remain.

Write:

$\nabla_{\theta} J = \nabla_{\theta} \int p_{\theta}(\tau) G(\tau) d\tau = \int p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau) G(\tau) d\tau$, $\log p_{\theta}(\tau) = \log p(s_0) + \sum_t [\log \pi_{\theta}(a_t | s_t) + \log p(s_{t+1} | s_t, a_t)] \Rightarrow \nabla_{\theta} J = \mathbb{E}_{\tau} [\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G_t]$.

Follow-up: “Why a baseline b ?” Subtracting $b(s_t)$ does not change the expected gradient but lowers its variance.

Picture it. Training a dog: after a good walk (high return), make every action of that walk a bit more likely; after a bad one, a bit less.

Q6.6 Confusion matrix, precision, recall, AUC.

general

Say: Precision = $TP/(TP+FP)$: of what I called positive, how much is right. Recall = $TP/(TP+FN)$: of all positives, how many I found. The ROC curve plots true-positive rate against false-positive rate over all thresholds; AUC is the probability that a random positive scores higher

than a random negative (0.5 chance, 1 perfect).

7 rCFD and CFD (Prof. Pirker)

2-minute summary ■ rCFD in two minutes

One idea. Solve the expensive flow once, record *how the fluid moves* as cell-to-cell maps, then replay those maps many times to carry a scalar (species, temperature). Momentum and pressure are never solved again: hundreds of times faster.

$$T_j^* = \sum_i M_{ji}^{(f)} T_i, \quad \mathbf{T}^{n+1} = \mathcal{B}(\mathcal{C}(\mathcal{S}(\mathbf{M}^{(f_n)} \mathbf{T}^n))), \quad R_{fg} = \|\Phi^{(f)} - \Phi^{(g)}\|.$$

map $\mathbf{M}$ (advection) • face swap $\mathcal{S}$ (diffusion) • correction $\mathcal{C}$ (your work) • conservation constraint $\mathcal{B}$. Pirker's three bold failures: (1) mass imbalance → global monitor + mass-acting face swaps; (2) unphysical diffusion → locally adapted swaps (sub-grid viscosity); (3) weak recurrence prominence → recurrence islands. **Works:** recurring flow, passive scalar. **Fails:** the scalar drives the flow, or the process changes state → multi-state rCFD.

Picture it. A GPS that recorded one drive through the city. Next time you only replay the recorded route (cheap), you do not recompute it. It works as long as the streets are the same. If a street is closed (a new boundary condition, like the cooled lid), the recording does not know the detour: you need a new recording for that situation.

Board drill (write it by hand 3 times). $\mathbf{T}^{n+1} = \mathcal{B}(\mathcal{C}(\mathcal{S}(\mathbf{M}\mathbf{T}^n)))$ • $\Delta_{\text{swap}} = f_{\text{sd}} V (T_{c_1} - T_{c_0})/2$, $D_{\text{eq}} = f_{\text{sd}} c_s^2 / (2\Delta t)$ • limiter $\Delta q_f = s \min(Q_h^+, Q_c^-)$ • $Co = u\Delta t / \Delta x < 1$, $d = \Gamma\Delta t / \Delta x^2 < 0.5$ • $Re = uL/\nu$.

Q7.1 Flow-based versus transport-based rCFD?

Pirker

Say: Flow-based: stitch recorded flow sequences into a long artificial flow and solve a transport equation on it (Lichtenegger & Pirker 2016, about 100× faster). Transport-based: replace convection by cell-to-cell shifts and diffusion by face swaps, no equation solved (Pirker & Lichtenegger 2018, steel ladle 26 h → 1.6 s). My thesis uses the transport-based form.

Q7.2 Recurrence matrix and prominence – and your case?

Pirker

Say: The recurrence norm compares two recorded states; all pairs form the matrix. A row (recurrence vector) rises and oscillates about a plateau; deep dips = the flow comes back = we can jump. Prominence $P_{\text{rel}} = \langle d_{\text{max}} - d_{\text{min}} \rangle / \bar{d}$, required 0.05–0.1. Mine: median 0.028, two blocks (filling, settling), no return → sequential replay.

Q7.3 Why did the recorded local diffusivity not help?

Pirker

Say: The exchange is explicit and capped at half a cell per face and step, a diffusion number $Di = D\Delta t / c_s^2 \leq 0.5$. The recorded D asks for $Di \approx 4$ everywhere, so every face hits the cap and the field behaves like $f_{\text{sd}} = 0.5$ (three replays bit-identical). An implicit exchange or a smaller step would let it act. This is the explicit stability limit of the lecture, $d < 0.5$.

Q7.4 Numerical diffusion: where in your thesis?

Pirker

Say: First-order upwind is stable but adds false diffusion that smears fronts. In my replay the smearing comes from the face exchange, not the maps (maps only keep the front at 2–4 mm). My correction is an anti-diffusive step bounded like flux-corrected transport: conservative, no

new extremum, front $15.7 \rightarrow 5.3/4.5$ mm against 5.0 mm.

Q7.5 rCFD and digital twins.

Pirker

Say: CSTR models are instant but blind to heterogeneity; CFD sees it but takes days. rCFD built on CFD can run alongside the process as a soft sensor ($T_{\max}$, σ_T), aligned with live data to stop error accumulation. My thesis adds two warnings: a boundary condition can switch the flow regime, and the heat budget must use the same fluid properties as the recording.

Q7.6 CFD basics he may check quickly.

Pirker

Say: $Re = uL/\nu$ (mine 99, laminar). Material derivative $D/Dt = \partial_t + \mathbf{u} \cdot \nabla$. FVM: integrate over cells, fluxes through faces, locally conservative, $a_{PCP} = \sum a_{NBCNB} + b$. SIMPLE: guess p , solve $\mathbf{u}^*$, pressure correction from continuity, correct, repeat. Turbulence: cascade to Kolmogorov $\eta = (\nu^3/\varepsilon)^{1/4}$; RANS models all, LES the small eddies, DNS none; $\nu_t = C_\mu k^2/\varepsilon$. Boussinesq: constant density except in the gravity term (mine: 1.5 per cent density change).

8 Predicted questions and rapid-fire drill

8.1 Thesis questions (first 5 minutes)

2-minute summary ■ Your thesis in two minutes

rCFD records a flow once and replays it to carry a scalar. It was built for passive scalars; I tested it on temperature, which drives its own flow (buoyant jet in a $75 \times 75 \times 15$ mm tank, Re 99). (1) No single mixing constant is best: the outlet sits under the front, so smearing drains heat. (2) A bounded, vertically weighted anti-diffusive correction brings the front from 15.7 to 5.3/4.5 mm (reference 5.0), no value outside the data range. (3) Under wall cooling the replay was 4.34 K too warm because the reference used c_p of air (1006.43) and the replay of water (4182); on one fluid 0.66 K, corrected 0.06 K on 9.62 K. (4) What remains is missing transport: a recorded flux cuts the heterogeneity error from 1.66 to 0.26 K. Take-home: a database belongs to a flow regime.

Picture it. A cooking video recorded once (preparation) and replayed to cook many times (replay). It works for soup that just gets stirred (passive scalar). For bread that rises by itself (buoyancy), the video does not show how your dough will move, so you must correct it.

Q8.1 What problem did you solve, and why ML/AI?

chair

Say: I measured what a recurrence replay loses when the scalar is not passive, and repaired most of it without solving the flow again. It is AI for simulation: expensive solver data is reused by a cheap model; choosing the next frame is a nearest-neighbour search; and my main result is a generalization result: the database covers a regime, and a new boundary condition is out-of-distribution.

Q8.2 What would you do differently?

chair

Say: Check the material properties of both simulations on the first day: the specific-heat mismatch cost the most time. Then an implicit exchange so the recorded diffusivity can act.

Q8.3 How would you solve your problem with deep learning?

Klambauer

Say: Learn the correction $\mathcal{C}$ with a graph neural network on the mesh: input the field and the recorded map, output the next field, loss = distance to the reference plus a penalty on the energy balance. The risk is the same as in my thesis: it only generalizes inside the regime of its training

data.

Q8.4 Could an LSTM replace the replay?

Hochreiter

Say: An LSTM (or xLSTM) could learn the time evolution of a low-dimensional state, e.g. the POD coefficients of Figure 5.9, as a sequence-to-sequence model. It would be cheap and could carry memory of the filling phase, which is exactly where my replay lags. But it needs many trajectories to generalize, and without a conservation constraint it can drift in energy, which rCFD guarantees by construction.

8.2 Rapid-fire (say it in one breath)

Question	Answer
Most powerful learning machine?	The brain; closest method: neural networks.
Delta, softmax + CE?	$\hat{\mathbf{y}} - \mathbf{y}$.
Activation from Linz?	SELU (self-normalizing), ELU.
Who found vanishing gradients?	Hochreiter, 1991, Linz.
Constant error carousel?	Cell self-loop weight 1, linear: error flows unchanged.
Forget gate added by?	Gers, Schmidhuber, Cummins, 2000.
GRU gates?	Update (coupled input/forget) and reset.
BPTT memory?	Linear in T ; RTRL: independent of T , $O(N^4)$ per step.
Truncated BPTT?	Cuts the past after k steps; approximate gradient.
Gradient clipping fixes?	Exploding only, not vanishing.
Why $\sqrt{d_k}$?	Keeps dot-product variance at 1; softmax does not saturate.
Attention complexity?	$O(N^2)$; RNN $O(N)$.
Positional encoding why?	Attention is permutation-equivariant.
xLSTM two ideas?	Exponential gating; matrix memory (mLSTM).
Hopfield $\leftrightarrow$ attention?	$\mathbf{X} \text{softmax}(\beta \mathbf{X}^\top \boldsymbol{\xi}) = \text{attention}$.
UAT hidden layers?	One, finitely many units.
Non-differentiable loss?	0–1 loss; use cross-entropy or hinge.
Regularization?	Weight decay, dropout, early stopping, augmentation, FMS.
Optimizers?	SGD, momentum, AdaGrad, RMSprop, Adam.
Output activation, regression?	Linear + squared loss.
VC dim. of lines in 2D?	3.
Kernel trick?	Dot products $\rightarrow k(\mathbf{x}, \mathbf{x}')$.
MLE problem?	Overfits with little data (sunrise problem).
Your Re / grid / frames?	99 / 1 mm cubes / 600 frames.
Your speed-up?	About 260 times.